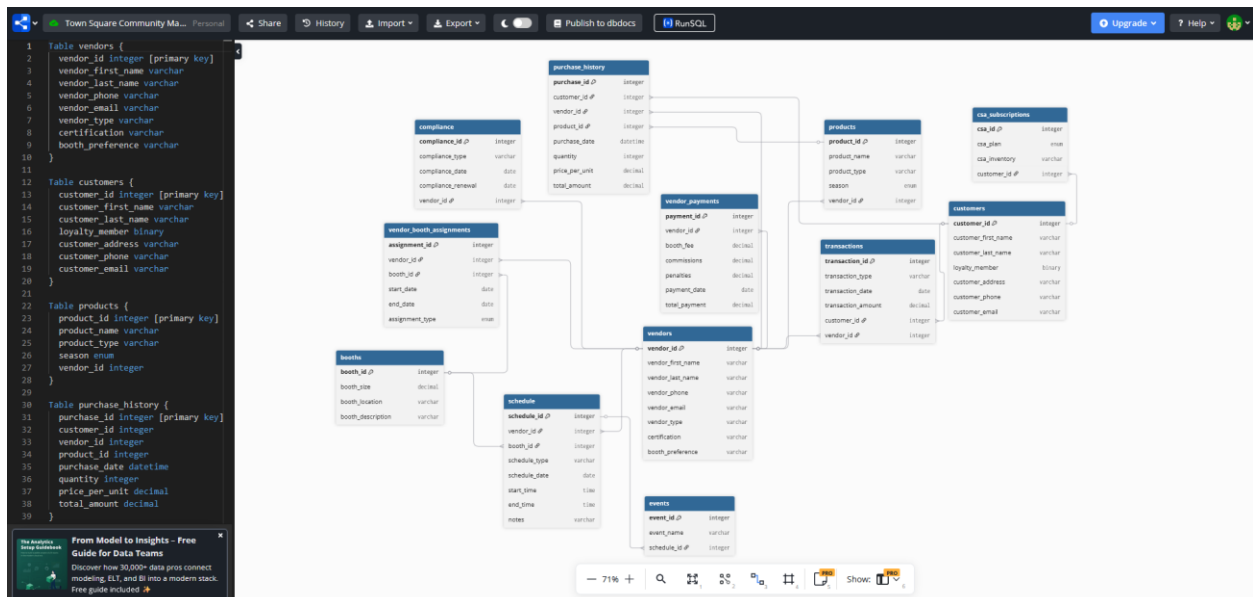


Introduction

The business for which my database was created is Town Square Community Market, a local farmers market where farmers, food cooperatives, craftspeople, and artisans can sell their produce and goods directly to the community. The market is a vendor-based business that generates revenue through booth rental fees, vendor commissions, and supplementary services. It manages booth assignments through flexible permanent, seasonal, and temporary arrangements that accommodate natural agricultural production. Vendors undergo thorough vetting processes including background checks, and must maintain compliance with health department regulations, food handler licensing, liability insurance, and sales tax requirements. Many of the vendors hold specialized certifications such as organic, sustainable, or fair-trade. Customer engagement is prioritized through a loyalty program, purchase history tracking, and a Community Supported Agricultural (CSA) subscription service offering weekly and monthly delivery options with produce-only or comprehensive product selections. Town Square Community Market strives for community engagement by hosting educational events like cooking demonstrations and workshops in addition to supporting small-scale producers.

Entity Relationship Diagram (Database Schema)



[link to ERD](#)

The ERD for this database includes the entities vendors, customers, products, purchase history, booths, vendor booth assignments, schedule, events, vendor payments,

transactions, CSA subscriptions, and compliance. These entities interact with each other with the following relationships:

- Vendors can have multiple booth assignments and sell multiple products, which allows the market to track booth occupancy and available products.
- Purchase history connects customers, vendors, and products, tracking what customers buy from which vendors. Customers, vendors, and products can have multiple purchase histories.
- Each event has one schedule entry, but vendors and booths can appear in multiple schedule entries for different dates and events.
- Vendors can have multiple vendor payments because payments can happen multiple times for different reasons or at different times.
- Customers and vendors can have many transactions since they shop and sell one or more times.
- Customers can have zero to one CSA subscriptions.
- Vendors can have multiple compliance records to track permit renewals and regulatory requirements over time.
- Vendors can have multiple booth assignments that change depending on date or season, and booths can be assigned to different vendors at different times, allowing flexible space management.

Database Constraints

The Town Square Community Market database incorporates constraints to ensure data integrity and enforce critical business rules throughout the system. Primary key constraints using AUTO_INCREMENT provide unique identifiers for all entities, while foreign key constraints maintain referential integrity between related tables such as linking products to vendors, purchase history to customers and vendors, and booth assignments to both vendors and booths. Business logic is enforced through CHECK constraints that prevent invalid data entry, including ensuring positive quantities and unit prices in purchase history, non-negative values for booth fees, commissions, and penalties in vendor payments, valid date ranges where start dates precede end dates in booth assignments and schedules, and proper time sequences where start times occur before end times in scheduling. UNIQUE constraints prevent duplicate entries for critical business identifiers such as vendor phone numbers, email addresses, and business names, ensuring clear vendor identification and communication channels. The database also employs ENUM constraints to standardize categorical data like seasonal classifications for products (spring, summer, fall, winter), assignment types for booth rentals (permanent, seasonal,

temporary), and CSA subscription plans (weekly, monthly, produce-only, dry-goods-only). Additionally, NOT NULL constraints ensure that essential information such as vendor contact details, customer addresses, product names and types, and compliance dates are always provided.

Business Rules

The business rules for vendors are that each vendor must provide complete and unique contact information before being assigned a booth, and for communication and proper identification. Each vendor business name must be unique to prevent confusion in the market. Vendors can specify booth locations preferences, but assignments are subject to availability. Each vendor can hold multiple certifications (organic, fair-trade, etc.) to showcase their credentials.

Each booth can only be assigned to one vendor during any given time period to prevent double-booking. Booth assignments must have valid date ranges where the start date cannot be after the end date. Booth assignments are categorized as permanent, seasonal, or temporary to manage different vendor needs. Booths have specific characteristics (size, location, description) that determine their suitability for different vendors. The system tracks booth assignment history to support seasonal rotations and temporary arrangements.

Products are classified by season (spring, summer, fall, or winter) to reflect agricultural cycles and availability. Each product must be associated with a specific vendor to maintain accountability and track transactions. Products must have both a name and type classification for categorization. Vendors can offer multiple products, allowing for diverse product portfolios.

All purchase transactions must have positive quantities and unit prices to ensure valid sales. The total amount for each purchase must equal quantity multiplied by price per unit to maintain financial accuracy. Vendor payments include booth fees, commissions, and penalties, all of which must be non-negative values. All financial transactions are tracked with dates to support financial reporting and analysis.

Customers must provide their complete address, phone, and email for communication and deliveries. Customer loyalty membership status is tracked to support the rewards program. Each customer can have at most one active CSA subscription to simplify subscription management. CSA subscriptions are categorized by frequency (weekly, monthly) and type (produce-only, dry-goods only).

All scheduled activities must have valid time ranges where start time happens before end time. Market operating hours and vendor schedules are tracked separately to manage different operational needs. Special events are linked to specific schedule entries to coordinate vendor participation. Schedule entries can be associated with specific vendors and booths for targeted activities.

Vendors must maintain current compliance records for health permits, licenses, and insurance. Each compliance record has both an initial compliance date and renewal date to track expiration. Multiple compliance types can be tracked per vendor (health permits, food handler licenses, liability insurance, sales tax permits). The system monitors upcoming renewal dates to ensure continuous regularity compliance.

All foreign key relationships are enforced to maintain data consistency across related entities. Vendor booth assignments reference valid vendors and booths to prevent orphaned records. Purchase history links valid customers, vendors, and products to ensure transaction accuracy. Schedule entries reference existing vendors and booths when applicable. Payment records are linked to specific vendors and booths to maintain financial accountability.

The database uses indexes on frequently queried fields (vendor IDs, customer IDs, dates, email addresses) to optimize performance. Views are provided for common queries like active CSA subscriptions and upcoming compliance renewals. A stored procedure validated booth availability before making new assignments to prevent conflicts. The system supports flexible date ranges for temporary and seasonal arrangements while maintaining permanent assignments.

Database Normalization

1st normal form

My database design conforms to 1st normal form by not having any repeating values. Each column contains a single scalar value. For example, the schedule table contains separate columns for date, start time, and end time. Each of these contains a single, specific entry. They do not contain lists or composite values, and there are no repeating groups or redundant values. For example, in my schedule table I chose to make separate columns for date, start time, and end time. Because some schedules repeat and may occur daily,

and some schedules are events that only happen once, I chose to do this instead of having a single datetime value for each event, which would cause repeating values for the schedules that occur often. Additionally, fields like certification in the vendors table store single values (e.g. “organic”) rather than lists like “organic, fair-trade, sustainable” which would violate 1NF. Each table has a primary key, which ensures row uniqueness and prevents data duplication in the rows. All the foreign keys are linked properly, which avoids duplication and makes sure they are referencing and not creating new data.

2nd normal form

The database is in 2nd normal form because all of the non-key columns depend on the primary key. For example, I have a separate events table outside the schedule table. If event ID and event name were included in the schedule table, they wouldn’t entirely depend on the primary key of schedule ID. It would cause redundant data on the schedule table because you’d have to put in null values for every schedule item that isn’t a singular event. Similarly, in the purchase history table, all columns (purchase date, quantity, price per unit) depend on the full composite key of customer ID, vendor ID, and product ID, not just part of it. This eliminates partial dependencies.

3rd normal form

My database conforms to 3rd normal form by having every non-key column depend only on the primary key. For example, all the columns in the vendors table (vendor first name, vendor last name, vendor phone, etc.) depend only on vendor ID. Booth preference sounds like it could be in the booths column, but since it’s vendors who choose their booth preference, it depends on vendor, not booth. I also avoided storing booth location in the vendors payments table, even though payments are related to booths. Instead, booth location depends only on booth ID in the booths table, eliminating transitive dependencies where vendor payment -> booth ID -> booth location. This normalization reduces data redundancy, prevents update anomalies, and ensures data integrity throughout the farmers market database.

Backup Strategy

The backup strategy for my database is a cronjob that backs up daily at midnight. It does a full database dump of the tscm database, and names each backup with the full date it

happened on. The backup method is a MySQL dump using mysqldump utility. The storage location is in the /backups/ directory. Backups can be restored by restoring the entire database, extracting specific tables from the backup, or with backups of specific dates when the database was backed up.

```
0 0 * * * /usr/bin/mysqldump -u tscm_admin -p'WalPhoCom29?' tscm >
~/backups/tscm_full_$(date +%F).sql
```

User Accounts

Admin user account

Can manage all tables, users, and permissions.

```
CREATE USER 'tscm_admin'@'20.124.88.14' IDENTIFIED BY '*****';
```

```
GRANT ALL PRIVILEGES ON tscm.* TO 'tscm_admin'@'20.124.88.14';
```

Market manager user account

Has access to add and delete records to the tscm database. They have database privilege level.

```
CREATE USER 'tscm_manager'@'20.124.88.14' IDENTIFIED BY '*****';
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON tscm.* TO 'tscm_manager'@'20.124.88.14';
```

Customer service account

Has privileges to view and update customer and subscription data

```
CREATE USER 'tscm_customer_service'@'20.124.88.14' IDENTIFIED BY '*****';
```

```
GRANT SELECT, INSERT, UPDATE ON tscm.customers TO
'tscm_customer_service'@'20.124.88.14';
```

```
GRANT SELECT, INSERT, UPDATE ON tscm.csa_subscriptions TO
'tscm_customer_service'@'20.124.88.14';
```

Read-only account

[phpmyadmin link](#)

Has privilege to just view database, for Mark Locklear

```
CREATE USER 'tscm_viewer'@'%' IDENTIFIED BY 'DesToiWin16!';
```

```
GRANT SELECT ON tscm.* TO 'tscm_viewer'@'%';
```

How I implemented md5

```
CREATE TABLE tscm_users (
```

```
user_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
username VARCHAR(50) UNIQUE NOT NULL,
```

```
password_hash CHAR(64) NOT NULL,
```

```
role ENUM('admin', 'manager', 'customer_service', 'read_only') NOT NULL
```

```
);
```

```
INSERT INTO tscm_users (username, password_hash, role) VALUES ('tscm_admin',  
SHA2('*****', 256), 'admin');
```

```
INSERT INTO tscm_users (username, password_hash, role) VALUES ('tscm_manager',  
SHA2('*****', 256), 'manager')
```

```
INSERT INTO tscm_users (username, password_hash, role) VALUES  
('tscm_customer_service', SHA2('*****', 256), 'customer_service');
```

```
INSERT INTO tscm_users (username, password_hash, role) VALUES ('tscm_read_only',  
SHA2('DesToiWin16!', 256), 'read_only');
```

Server Monitoring Strategy

To monitor the performance and health of the MySQL server hosting my database, I will track performance metrics including query response times, CPU and memory usage, disk space utilization, and active connection counts. Health metrics will include server uptime, error rates, failed connections, and backup completion status.

My monitoring tools will be MySQL Workbench for real-time performance dashboards and MySQL Error/Slow Query Logs for identifying problematic queries. I will set alert thresholds at 85% CPU usage, 90% disk space, and 5-second query response times, with daily review of logs and weekly performance trend analysis.